

```

import numpy as np
import matplotlib.pyplot as plt

print("=====")
print("    WIND FARM LAYOUT GENETIC ALGORITHM CONFIG    ")
print("=====")

try:
    ACRES_INPUT = float(input("Enter Wind Farm Target Size (in Acres,
e.g., 1200): "))
    ROTOR_RADIUS = float(input("Enter Turbine Rotor Radius (in Meters,
e.g., 40): "))
    SAFETY_MULTIPLIER = float(input("Enter Separation Distance
Multiplier (e.g., 5 for 5D spacing): "))
    V0 = float(input("Enter Free-Stream Ambient Wind Speed (in m/s,
e.g., 12): "))
    WIND_DEG = float(input("Enter Wind Direction Angle (in Degrees,
0=N, 90=E, 180=S, 270=W): "))
except ValueError:
    print("\n[Error] Invalid numerical input! Falling back to standard
default values.")
    ACRES_INPUT = 1200.0
    ROTOR_RADIUS = 40.0
    SAFETY_MULTIPLIER = 5.0
    V0 = 12.0
    WIND_DEG = 90.0

WIND_RAD = np.radians(270.0 - WIND_DEG)
WIND_DIR_X = np.cos(WIND_RAD)
WIND_DIR_Y = np.sin(WIND_RAD)

TOTAL_AREA_M2 = ACRES_INPUT * 4046.86
WIND_FARM_DIM = np.sqrt(TOTAL_AREA_M2)
WIND_FARM_WIDTH = WIND_FARM_DIM
WIND_FARM_HEIGHT = WIND_FARM_DIM

CELL_SIZE = SAFETY_MULTIPLIER * (2.0 * ROTOR_RADIUS)

N_COLS = max(2, int(WIND_FARM_WIDTH // CELL_SIZE))

```

```

N_ROWS = max(2, int(WIND_FARM_HEIGHT // CELL_SIZE))
CHROMOSOME_LEN = N_ROWS * N_COLS

print("\n" + "="*50)
print(f"--- Configured Framework from Converted User Input ---")
print(f"Input Target Size : {ACRES_INPUT:.1f} Acres")
print(f"Equivalent Boundary: {WIND_FARM_WIDTH:.1f}m x {WIND_FARM_HEIGHT:.1f}m")
print(f"Safety Step Buffer : {CELL_SIZE:.1f} meters")
print(f"Grid Framework : {N_ROWS} Rows x {N_COLS} Columns ({CHROMOSOME_LEN} available nodes)")
print(f"Target Wind Vector : Speed = {V0:.1f} m/s | Angle = {WIND_DEG:.1f}°")
print("="*50 + "\n")

A_FACTOR = 0.333
WAKE_DECAY = 0.075
MAX_TURBINE_POWER = 2.0

def evaluate_layout(chromosome):
    turbines = []
    idx = 0
    for r in range(N_ROWS):
        for c in range(N_COLS):
            if chromosome[idx] == 1:
                turbines.append((c * CELL_SIZE, r * CELL_SIZE))
                idx += 1

    n_t = len(turbines)
    if n_t == 0:
        return 0.0, 0.0, 0.0, turbines

    sorted_indices = np.argsort([t[0]*WIND_DIR_X + t[1]*WIND_DIR_Y for t in turbines])
    turbines = [turbines[i] for i in sorted_indices]

    total_power = 0.0

    for i in range(n_t):
        xi, yi = turbines[i]
        max_deficit = 0.0

        for j in range(i):
            xj, yj = turbines[j]

            dx_raw = xi - xj

```

```

        dy_raw = yi - yj

        dx = dx_raw * WIND_DIR_X + dy_raw * WIND_DIR_Y
        dy = abs(-dx_raw * WIND_DIR_Y + dy_raw * WIND_DIR_X)

        if dx > 0.1:
            r_wake = ROTOR_RADIUS + WAKE_DECAY * dx
            if dy <= r_wake:
                deficit = (2.0 * A_FACTOR) / ((1.0 + WAKE_DECAY *
dx / ROTOR_RADIUS) ** 2)
                if deficit > max_deficit:
                    max_deficit = deficit

            v_effective = V0 * (1.0 - max_deficit)
            p_turbine = MAX_TURBINE_POWER * ((v_effective / V0) ** 3) if V0
> 0 else 0
            total_power += p_turbine

        theoretical_max = n_t * MAX_TURBINE_POWER
        efficiency = (total_power / theoretical_max) * 100 if
theoretical_max > 0 else 0

        cost = n_t * (1.0 + 0.005 * n_t)
        fitness = total_power / cost if cost > 0 else 0
        return fitness, total_power, efficiency, turbines

POP_SIZE = 40
GENERATIONS = 50
MUTATION_RATE = 0.04
ELITE_SIZE = 2

population = [np.random.choice([0, 1], size=CHROMOSOME_LEN, p=[0.85,
0.15]) for _ in range(POP_SIZE)]
history = []

print("Running Genetic Optimization Engine...")

for gen in range(GENERATIONS):
    generation_results = []
    for individual in population:
        fit, p_tot, eff, coords = evaluate_layout(individual)
        generation_results.append((fit, p_tot, eff, coords,
individual))

    generation_results = sorted(generation_results, key=lambda x: x[0],
reverse=True)

```

```

best_current = generation_results[0]
history.append((best_current[1], best_current[1], best_current[2]))

next_generation = [item[4] for item in
generation_results[:ELITE_SIZE]]

while len(next_generation) < POP_SIZE:
    contenders_1 = np.random.choice(POP_SIZE, 3, replace=False)
    parent_1 = generation_results[min(contenders_1)][4]

    contenders_2 = np.random.choice(POP_SIZE, 3, replace=False)
    parent_2 = generation_results[min(contenders_2)][4]

    cut = np.random.randint(1, CHROMOSOME_LEN - 1) if
CHROMOSOME_LEN > 2 else 1
    child = np.concatenate([parent_1[:cut], parent_2[cut:]])

    for bit in range(CHROMOSOME_LEN):
        if np.random.rand() < MUTATION_RATE:
            child[bit] = 1 - child[bit]

    next_generation.append(child)

population = next_generation
if (gen + 1) % 10 == 0 or gen == 0:
    print(f"-> Gen {gen+1:02d}/{GENERATIONS} | Top Power:
{best_current[1]:.2f} MW | Farm Efficiency: {best_current[2]:.1f}%")

best_fit, optimal_power, optimal_eff, optimal_coords, _ =
generation_results[0]

print("\n=====")
print("OPTIMIZATION ARCHITECTURE COMPLETE")
print("=====")
print(f"Optimal Asset Load-out : {len(optimal_coords)} Wind Turbines")
print(f"Total Farm Output      : {optimal_power:.2f} MW")
print(f"Calculated Efficiency    : {optimal_eff:.2f}%")
print("=====")

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

history = np.array(history)
ax1.plot(range(1, GENERATIONS + 1), history[:, 0], color='tab:blue',
lw=2)
ax1.set_title('GA Optimization Convergence Profile', fontsize=11,
fontweight='bold')
ax1.set_xlabel('Generations')
ax1.set_ylabel('Power Generation (MW)')

```

```

ax1.grid(True, linestyle=':')

x_coords = [t[0] for t in optimal_coords]
y_coords = [t[1] for t in optimal_coords]
ax2.scatter(x_coords, y_coords, color='teal', s=120, marker='1', lw=3,
label='Turbine Hub')

visual_dist = CELL_SIZE * 1.2
for x, y in optimal_coords:

    x_end = x + visual_dist * WIND_DIR_X
    y_end = y + visual_dist * WIND_DIR_Y

    r_wake_end = visual_dist * WAKE_DECAY
    perp_x = -WIND_DIR_Y * r_wake_end
    perp_y = WIND_DIR_X * r_wake_end

    ax2.plot([x, x_end + perp_x], [y, y_end + perp_y], color='red',
linestyle='--', alpha=0.15)
    ax2.plot([x, x_end - perp_x], [y, y_end - perp_y], color='red',
linestyle='--', alpha=0.15)

ax2.set_xlim(-100, WIND_FARM_WIDTH + 100)
ax2.set_ylim(-100, WIND_FARM_HEIGHT + 100)
ax2.set_title(f'Optimal Layout for {ACRES_INPUT:.1f} Acres Grid
({WIND_DEG}° Wind)', fontsize=11, fontweight='bold')
ax2.set_xlabel('Grid Width X (meters)')
ax2.set_ylabel('Grid Height Y (meters)')
ax2.grid(True, alpha=0.3)
ax2.legend()

plt.tight_layout()
plt.show()

```

OUTPUT:

